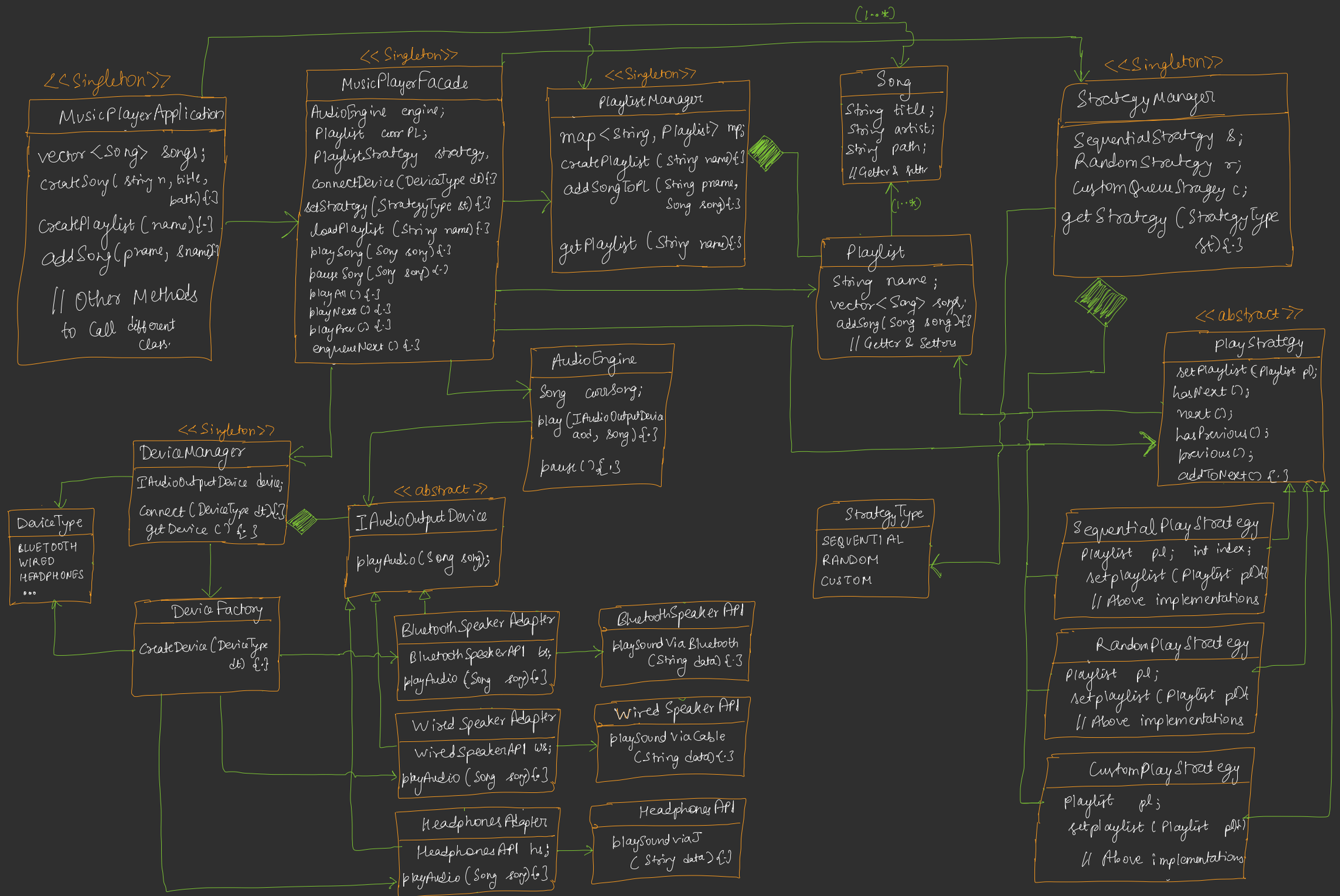

(1...*)
<<Singleton>>
MusicPlayerApplication
vector<Song> songs;
CreateSong(string n, title, path){.}
CreatePlaylist(name){.}
AddSong(pname, &name){.}
// Other Methods to Call different class.
<<Singleton>>
MusicPlayerFacade
AudioEngine engine;
Playlist curr PL;
PlaylistStrategy strategy;
ConnectDevice(DeviceType dt){.}
SetStrategy(StrategyType st){.}
loadPlaylist(string name){.}
playSong(Song song){.}
pauseSong(Song song){.}
playAll(){.}
playNext(){.}
playPrev(){.}
engineNext(){.}
<<Singleton>>
PlaylistManager
map<String, Playlist> mp;
CreatePlaylist(string name){.}
addSongToPL(string pname, Song song){.}
getPlaylist(string name){.}
Song
String title;
String artist;
String path;
// Getter & Setter
(1...*)
Playlist
String name;
vector<Song> songs;
addSong(Song song){.}
// Getter & Setter
<<Singleton>>
StrategyManager
SequentialStrategy s;
RandomStrategy r;
CustomQueueStrategy c;
getStrategy(StrategyType st){.}
<<abstract>>
PlayStrategy
setPlaylist(Playlist pl);
hasNext();
next();
hasPrevious();
previous();
addToNext(){.}
SequentialPlayStrategy
Playlist pl; int index;
setplaylist(Playlist pl){
// Above implementations
RandomPlayStrategy
Playlist pl;
setplaylist(Playlist pl){
// Above implementations
CustomPlayStrategy
Playlist pl;
setplaylist(Playlist pl){
// Above implementations
StrategyType
SEQUENTIAL
RANDOM
CUSTOM
AudioEngine
Song currSong;
play(IAudioOutputDevice aod, Song){.}
pause(){.}
<<Singleton>>
DeviceManager
IAudioOutputDevice device;
Connect(DeviceType dt){.}
getDevice(){.}
DeviceType
BLUETOOTH
WIRED
HEADPHONES
...
DeviceFactory
CreateDevice(DeviceType dt){.}
<<abstract>>
IAudioOutputDevice
playAudio(Song song);
BluetoothSpeakerAdapter
BluetoothSpeakerAPI bs;
playAudio(Song song){.}
BluetoothSpeakerAPI
playSoundViaBluetooth(String data){.}
WiredSpeakerAdapter
WiredSpeakerAPI ws;
playAudio(Song song){.}
WiredSpeakerAPI
playSoundViaCable(String data){.}
HeadphonesAdapter
HeadphonesAPI hs;
playAudio(Song song){.}
HeadphonesAPI
playSoundViaJ(String data){.}